I102 - Paradigmas de Programación

INTRODUCCIÓN A CONTAINERS

3.5

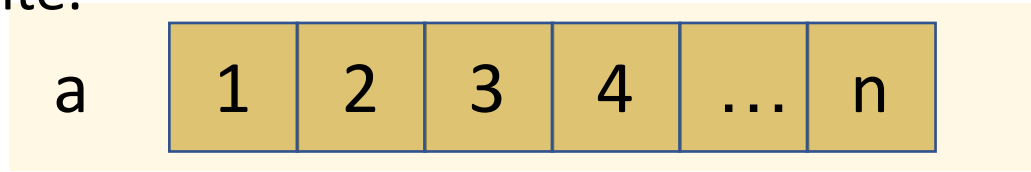
Prof. Mariano Scaramal

Ingeniería en Inteligencia Artificial

Contenedores Secuenciales

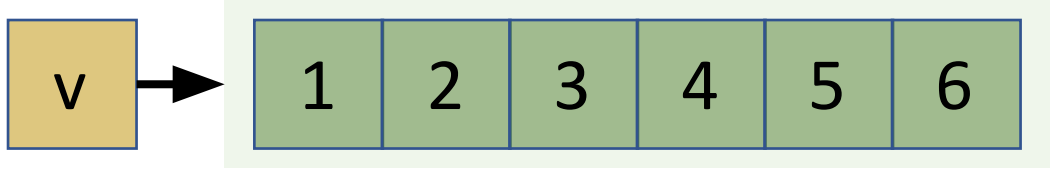
- Tamaño fijo, con datos almacenados en forma secuencial, no gestiona memoria dinámica internamente.

array<T, n>



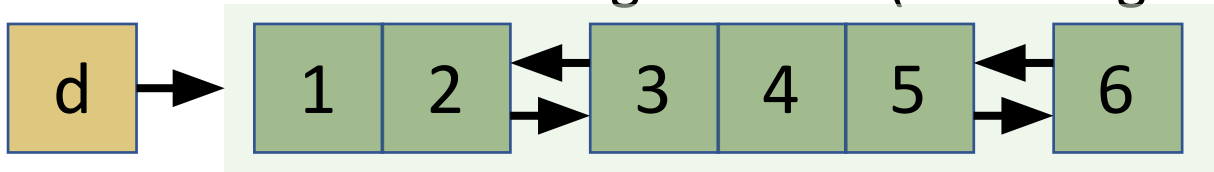
- Array de tamaño dinámico con datos almacenados en forma secuencial.

vector<T>



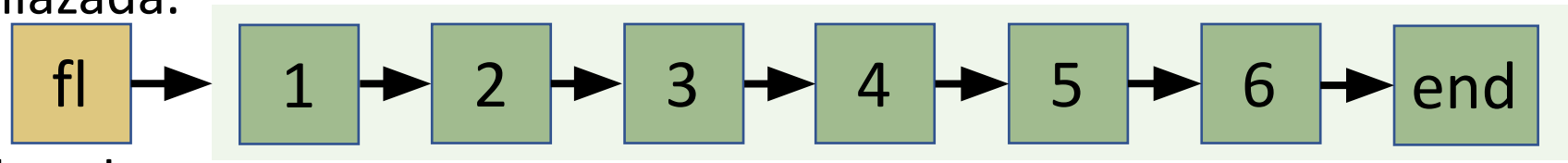
- Array de tamaño dinámico con estructura segmentada (no contigua).

deque<T>



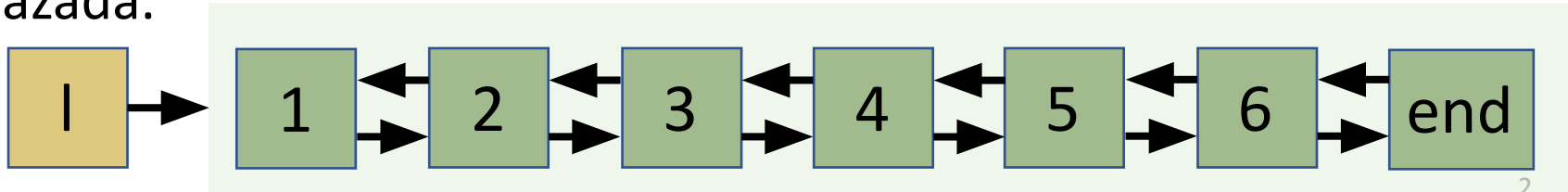
- Lista simplemente enlazada.

forward_list<T>



- Lista doblemente enlazada.

list<T>



Contenedores basados en índices

Operación	vector<T>	array<T, n>	deque<T>
Acceso	v[i] o v.at(i)	a[i] o a.at(i)	d[i] o d.at(i)
Agregar al final	v.push_back(val)	❌ (Tamaño fijo)	d.push_back(val)
Agregar al inicio	❌ (Es lento/no directo)	❌	d.push_front(val)
Insertar en medio	v.insert(it, val) (mueve elementos!)	❌	d.insert(it, val)
Borrar último	v.pop_back()	❌	d.pop_back()
Borrar específico	v.erase(it)	❌	d.erase(it)
Tamaño actual	v.size()	a.size()	d.size()
Limpiar todo	v.clear()	❌	d.clear()

Contenedores basados en nodos

Operación	list<T> (Doble)	forward_list<T> (Simple)
Acceso	Solo extremos (front(), back())	Solo el inicio (front())
Agregar al final	l.push_back(val)	❌ (sin puntero al último nodo)
Agregar al inicio	l.push_front(val)	fl.push_front(val)
Insertar en medio	l.insert(it, val)	fl.insert_after(it, val)
Borrar específico	l.erase(it)	fl.erase_after(it)
Tamaño actual	l.size()	❌ (antes de C++17)

Array (std::array)

Este container funciona en forma similar al array ya conocido de C.

Al igual que en C, *std::array* requiere un tamaño fijo.

Debido a esto, no se aloca memoria en tiempo de ejecución.

Si bien en el array de C también se pueden almacenar objetos, *std::array* tiene funcionalidades útiles para el manejo de objetos.

```
#include <iostream>
#include <array>

int main() {
    std::array<int, 5> arr = {3, 2, 6, 1, 5};
    // Acceder valores como un array de C y con .at()
    std::cout << "El primer valor es: " << arr[0] << std::endl;
    std::cout << "El cuarto valor es: " << arr.at(3) << std::endl;

    // Utilizar el for por rango
    std::cout << "Los elementos de arr son: ";
    for (const int n : arr)
        std::cout << n << " ";
    std::cout << std::endl;

    return 0;
}
```

Vector (std::vector)

Este container encapsula un array de tamaño dinámico.

Debido a su versatilidad, suele utilizarse más frecuentemente que los arrays además de poseer varios métodos interesantes como ser insert, erase, clear, reverse, empty, capacity, at, etc.

```
#include <iostream>
#include <string>
#include <vector>

using namespace std;

void printVec(vector<int> v, string name){
    cout << name + " es: ";
    for (const int n : v)
        cout << n << " ";
    cout << endl;
}
```

```
int main() {
    vector<int> vec1 {1, 2, 3};
    vector<int> vec2 (4, 9);
    // Imprime los vectores
    printVec(vec1, "vec1"); printVec(vec2, "vec2");
    // Uso de push_back y pop_back
    vec1.push_back(4); vec2.pop_back();
    // Uso de size(), front() y back()
    cout << "Vec1 tiene un size de " << vec1.size() << endl;
    cout << "Vec1 comienza con " << vec1.front() << endl;
    cout << "Vec1 termina con " << vec1.back() << endl;
    // Imprime nuevamente con las modificaciones
    printVec(vec1, "vec1"); printVec(vec2, "vec2");

    return 0;
}
```

Cola de doble extremo (std::deque)

Es una cola donde se puede agregar y sacar información del frente o el final.

Al igual que std::vector, std::deque también tiene los métodos front, back, at, begin, size, empty, insert y erase (no tiene capacity), y más.

```
#include <iostream>
#include <deque>
using namespace std;

void printDeq(deque<int> d, string name){
   cout << name + " es: ";
   for (const int n : d)
      cout << n << " ";
   cout << endl;
}
```

```
int main() {
   deque<int> Dq;
   // Agregar elementos al frente y al final
   Dq.push_back(3);   // Agrega al final
   Dq.push_back(8);   // Agrega al final
   Dq.push_front(5);  // Agrega al frente
   Dq.push_front(1);  // Agrega al frente
   // Se imprime con los datos agregados
   printDeq(Dq, "Dq");
   // Se remueve información
   Dq.pop_front();  // Remueve el primer elemento
   Dq.pop_back();   // Remueve el último elemento
   // Se imprime luego de las modificaciones
   printDeq(Dq, "Dq");

   return 0;
}
```

Listas (std::forward_list y std::list)

Implementan una lista simplemente enlazada (std::forward_list) y una lista doblemente enlazada (std::list). No se provee acceso aleatorio en ningún caso.

Tienen métodos similares a std::vector y los propios según el tipo de lista.

Ejemplo con forward_list

```
#include <iostream>
#include <forward_list>
using namespace std;

void printFL(forward_list<int> fl, string name){
    cout << name + " es: ";
    for (const int n : fl)
        cout << n << " ";
    cout << endl;
}
```

```
int main() {
    forward_list<int> myList = {5, 8, 3, 4, 1, 6};
    // Imprimir lista
    printFL(myList, "myList");
    myList.push_front(0); // Insertar al frente
    myList.push_front(9); // Insertar al frente
    myList.remove(3);      // Remover todos los 3
    // Imprimir luego de las modificaciones
    printFL(myList, "myList");
    myList.reverse(); // Invertir la lista
    // Imprimir lista invertida
    printFL(myList, "myList");

    return 0;
}
```

Pilas y Colas

Las pilas y colas son implementadas con las librerías *stack* y *queue*, respectivamente.

En las pilas, se tienen los métodos *push* y *pop*, propios de este modelo. Nótese que se utiliza el modelo LIFO para este caso.

En las colas, también se tienen los métodos *push* y *pop* que funcionan con el modelo FIFO, típico de una cola.

Las pilas y las colas se pueden inicializar con *push*, con un *vector* o con una *list*.

Recuerden: .pop() elimina el elemento del contenedor, mientras que .top() (en stack) y .front() (en queue) los devuelven sin remover del contenedor.

```
#include <iostream>
#include <stack>
#include <queue>
#include <vector>
#include <list>

int main() {
    std::vector<int> vec = {1, 2, 3};
    std::list<int> lst = {10, 20, 30};

    std::stack<int, std::vector<int>> st(vec);
    std::stack<int, std::list<int>> st2(lst);
    std::queue<int, std::vector<int>> q(vec);
    std::queue<int, std::list<int>> q2(lst);

    std::cout << st.top() << std::endl;     // Imprime 3
    std::cout << st2.top() << std::endl;    // Imprime 30
    std::cout << q.front() << std::endl;    // Imprime 1
    std::cout << q2.front() << std::endl; // Imprime 10

    return 0;
}
```

Pilas y Colas

Característica	stack	queue
Tipo de estructura	Pila	Cola
Orden	LIFO (Last In, First Out)	FIFO (First In, First Out)
Inserción	push(x) (arriba)	push(x) (final)
Eliminación	pop() (arriba, elimina elemento)	pop() (frente, elimina elemento)
Ver elemento	top()	front()
Acceso por índice	❌ No	❌ No
Vacío	empty()	empty()
Tamaño	size()	size()

Pilas y Colas - Ejemplos con cont. default

```cpp
#include <iostream>
#include <stack>

int main() {
    std::stack<int> s;

    for (int i = 1; i <= 5; i++)
        s.push(i);

    while (!s.empty()) {
        std::cout << s.top() << " ";
        s.pop();
    }

    return 0;
}
```

Pila

```cpp
#include <iostream>
#include <queue>

int main() {
   std::queue<int> q;

   q.push(10);
   q.push(20);
   q.push(30);
   std::cout << q.front() << std::endl;//10

   q.pop();
   std::cout << q.front() << std::endl;//20

    return 0;
}
```

Cola

Set y Multiset

El *std::set* es un conjunto de elementos que no puede tener repetidos.

Por otro lado, el *std::multiset* es un conjunto de elementos que pueden ser repetidos.

Ambos containers mantienen los elementos ordenados. De usar valores numéricos el default es en orden ascendente.

```
int x = 5;
auto it = setDes.find(x);// Busca un element en el container
if (it != setDes.end())     // iterador será .end() si no lo encuentra
   std::cout << "Encontrado: " << *it;  // Se recupera con *it
else
   std::cout << "No encontrado";
```

```
#include <iostream>
#include <set>
#include <string>

using namespace std;

int main() {
   set<int> setAsc = {5, 2, 8, 3, 10};
   multiset<int, greater<int>> setDes = {3, 5, 9, 21, 5};

   cout << "setAsc es: ";
   for (const auto val : setAsc)
      cout << val << " ";       // Imprime 2 3 5 8 10
   cout << std::endl;

   cout << "setDes es: ";
   for (const auto val : setDes)
      cout << val << " ";       // Imprime 21 9 5 5 3
   cout << std::endl;

   return 0;
}
```

Set y Multiset

Característica / Operación	`set<T>`	`multiset<T>`
Elementos	únicos ❗	repetidos permitidos
Orden	ordenado	ordenado
Inserción	`insert(x)`	`insert(x)`
Inserta duplicados	❌ ignorados	✅ permitidos
Búsqueda	`find(x)`	`find(x)`
Cantidad	`count(x)` → 0 o 1	`count(x)` → ≥ 0
Eliminar por valor	`erase(x)` → 0 o 1	`erase(x)` → elimina todos los x
Acceso por índice	❌ no	❌ no

Map y Multimap

El *std::map* asocia un *key* con un *value*. Similarmente a set, no permite duplicados.

Por otro lado, un *multimap* permite elementos duplicados.

Cada elemento de un map/multimap es un element std::pair.

El elemento *std::pair* es de la librería <utility> (aca tanto std::pair como *std::make_pair* están en <map>) y simplemente crea una asociación de par entre dos elementos.

```
#include <iostream>
#include <string>
#include <map>
using namespace std;

int main() {
    map<string, int> m; pair<string, int> single_elem;
    // Producto y precio por kg
    m["manzana"] = 5; m["banana"] = 2;
    single_elem = {"uva", 7};
    // Para insertar un elemento se puede hacer con un pair
    m.insert(make_pair("sandía", 1));
    m.insert(single_elem);
    // Para recorrerlo
    for (const auto& pair : m)
        cout << pair.first << ": " << pair.second << endl;
    // También se pueden acceder los precios
    cout << "El precio de la banana es: " << m["banana"] << endl;
    // Borrar un elemento
    m.erase("banana");
    return 0;
}
```

Map y Multimap

Característica / Operación	`map<K,V>`	`multimap<K,V>`
Claves	únicas	repetidas permitidas
Orden	por clave	por clave
Inserción	`m[k] = v` ó `insert()`	solo `insert()`
Clave duplicada	❌ sobrescribe	✅ agrega otra
Acceso directo	`m[k]`	❌ no existe
Acceso seguro	`at(k)`	❌ no
Búsqueda	`find(k)`	`find(k)`
Cantidad	`count(k)` → 0 o 1	`count(k)` → ≥ 0
Eliminar por clave	`erase(k)` → 0 o 1	`erase(k)` → elimina todos

Iteradores

Proporciona una interfaz uniforme para acceder a los elementos, independientemente de cómo estén almacenados internamente (vector, list, set, etc.).

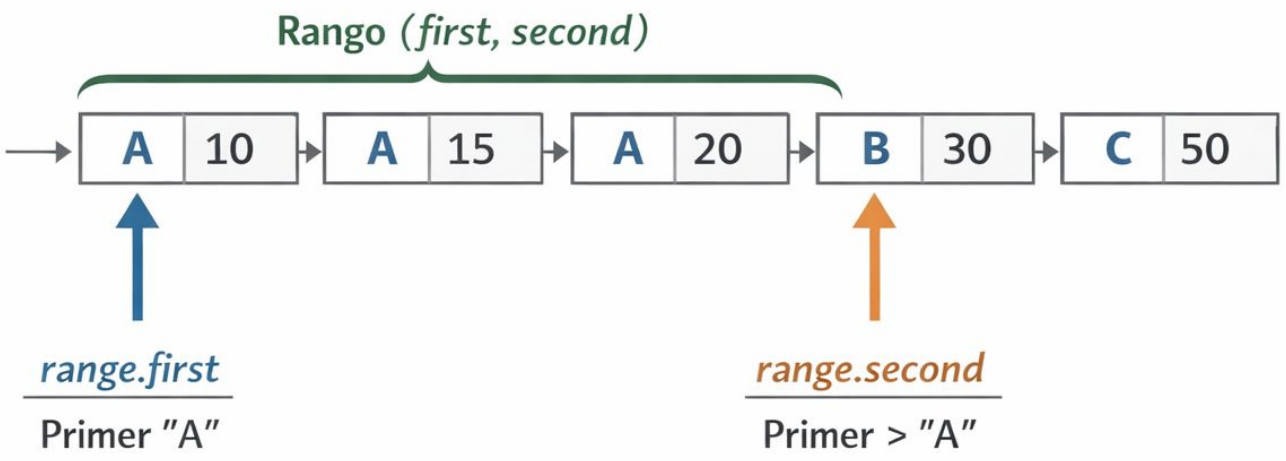
Es decir, el código que recorre elementos no necesita saber si el contenedor es contiguo, enlazado o basado en árbol. No expone la estructura interna.

Funciona en forma similar a un puntero, pero el manejo de memoria lo hace el container por lo que no es necesario el liberar la memoria.

```
#include <iostream>
#include <set>

int main() {
   std::multiset<int> ms = {10, 20, 20, 30};
   // Recorre todo el multiset
   std::cout << "Multiset completo usando iterador: ";
   for (std::multiset<int>::iterator it = ms.begin(); it != ms.end(); ++it)
      std::cout << *it << " "; // imprime: 10 20 30
   std::cout << "\n";

   // Devuelve un par de iteradores al rango de elementos iguales
   auto range = ms.equal_range(20);
   std::cout << "Rango de elementos iguales (20) en multiset: ";
   for (auto it = range.first; it != range.second; ++it)
      std::cout << *it << " "; // imprime: 20 20
   std::cout << "\n";

   return 0;
}
```

Iteradores - range_equal

```
auto range = mm.equal_range("A");
```



Iterar desde range.*first* hasta range.*second*

Modificar un Elemento de un Container

Se puede utilizar un iterador o un *for* para modificar un valor en un contenedor, pero depende de cual sea el contenedor.

Si no tiene en cuenta el orden se puede hacer directamente, sino hay que eliminarlo y luego insertar el nuevo valor para que sea ordenado.

```
#include <iostream>
#include <vector>
#include <set>

int main() {
   std::vector<int> v = {1, 2, 3};
   std::cout << "Vector original: ";
   for (auto it = v.begin(); it != v.end(); ++it){
      std::cout << *it << " "; // imprime: 1 2 3
      *it *= 2;       // modifica el valor directamente
   }
   std::cout << "\nVector modificado: ";
   for (auto x : v) std::cout << x << " ";
   std::cout << "\n\n";

   std::set<int> s = {10, 20, 30};
   std::cout << "Set original: ";
   for (auto x : s) std::cout << x << " ";
   std::cout << "\n";

   auto it = s.find(20);
   if (it != s.end()) {// elimina elemento e inserta el nuevo
      s.erase(it); s.insert(25);
   }

   std::cout << "Set modificado (20 -> 25): ";
   for (auto x : s) std::cout << x << " ";
   std::cout << "\n";

   return 0;
}
```

Preguntas?